

Laboratory 9:

Continuous-Time Fourier Transform & Convolution Real-Time FIR Audio Filter Design

Model-Based Design with Simulink and Auto-Generated Code

Lab Duration: 2 hours

Pre-Lab Time Expected: 30 minutes (filter coefficient design and hand calculations)

Key Concept: Professional DSP engineers use Model-Based Design (Simulink) to build real-time filters graphically and auto-generate microcontroller code. You will design a Finite Impulse Response (FIR) filter and validate it in hardware.

Contents

1	Real-World Context: FIR Filters Drive Consumer Electronics	4
1.1	Program 1: Voice-Activated Smart Audio Devices (Consumer Electronics)	4
1.2	Program 2: Cellular Modem Equalizers (Wireless Communications)	4
1.3	Program 3: Medical EEG Monitoring for Epilepsy Detection (Biomedical Devices) .	4
1.4	The Common Foundation: Model-Based Design with Simulink	5
2	Learning Objectives	6
3	Core Concepts: FIR Filters and the Fourier Transform	7
3.1	The Ideal Low-Pass Filter and Its Impulse Response	7
3.2	Finite Impulse Response (FIR) Filters	7
3.3	Frequency Response of an FIR Filter	8
3.4	Convolution in the Frequency Domain	8
4	Required Equipment and Software	9
5	Safety and Critical Notes	10
6	Hardware Setup: The Audio Front-End	11
6.1	Signal Path Overview	11
6.2	Microphone Preamplifier Circuit	11
6.3	Arduino Connections	11
6.4	RC Low-Pass Filter on PWM Output	11
6.5	Oscilloscope Setup	12
7	Pre-Lab Activity: Theoretical Filter Design	13
7.1	Assignment: Your Unique Cutoff Frequency	13
7.2	Task 1: Hand-Derive the Ideal Sinc Impulse Response	13
7.3	Task 2: List Your Windowed FIR Coefficients	14
7.4	Task 3: Compare with MATLAB's <code>fir1()</code>	14
7.5	Task 4: Compare Hand vs. MATLAB	15

8	Software Implementation: Simulink Model-Based Design	17
8.1	Overview of the Skeleton Model	17
8.2	Step 1: Prepare Your MATLAB Workspace	17
8.3	Step 2: Open the Skeleton Simulink Model	17
8.4	Step 3: Insert the Discrete FIR Filter Block	17
8.5	Step 4: Load Coefficients from Workspace	18
8.6	Step 5: Build, Deploy & Start	18
9	Measurement & Verification: Empirical Frequency Response	20
9.1	Measurement Strategy	20
9.2	Equipment Configuration	20
9.3	Task 1: Measure Gain at Key Test Frequencies	20
9.4	Task 2: Capture Scope Waveforms at Key Frequencies	21
9.5	Task 3: Create the Empirical Bode Plot in MATLAB	21
9.6	Task 4: Analyze Agreement Between Theory and Hardware	22
10	Deliverables & TA Sign-Off	24
A	Fourier Transform Fundamentals	26
A.1	Continuous-Time Fourier Transform (CTFT)	26
A.2	The Ideal Low-Pass Filter	26
B	Discrete-Time Fourier Transform and FIR Filters	26
B.1	Discrete-Time Fourier Transform (DTFT)	26
B.2	FIR Filter Design: Window Method	26
C	MATLAB Code Templates and Examples	27
C.1	Complete Pre-Lab Workflow	27
D	Simulink and Model-Based Design Guide	28
D.1	Setting Up the Arduino Target	28
D.2	The Discrete FIR Filter Block	28
E	Troubleshooting Guide	28
E.1	Pre-Lab Issues	28
E.2	Simulink Deployment Issues	28
E.3	Measurement Issues	29
	References and Inspiration	30

Grading and Authenticity Requirements

Deliverable (1/2): This is a graded laboratory assignment. Include your **name and student ID** in all MATLAB figures, Simulink screenshots, and scope captures.

Deliverable (2/2): Your hand-derived windowed-sinc filter coefficients must be shown to the TA and compared to MATLAB's `fir1()` output. Calculations must be in your own work; using online filter calculators is not acceptable.

Checkpoint (1/6): Before final submission, verify that each MATLAB plot includes proper axis labels, units, a legend, and your name/student ID in the figure title or `sgtitle`.

1 Real-World Context: FIR Filters Drive Consumer Electronics

You are a Junior DSP Engineer deployed across three product lines at your company. Each product depends on real-time FIR filters implemented in Simulink and auto-generated for embedded processors. Your role: design, validate, and ship these filters to millions of users worldwide.

1.1 Program 1: Voice-Activated Smart Audio Devices (Consumer Electronics)

Your company manufactures hearing aids, earbuds, and smart speakers with always-on microphone preprocessing. The challenge: raw microphone audio contains electrical hum (60 Hz), wind noise, background chatter, and rustling—all of which confuse downstream machine learning engines trained to recognize wake words.

The engineering problem: Voice intelligibility lives in the 85–4000 Hz band. Noise and artifacts above 4 kHz are useless for speech recognition and waste CPU cycles. But you cannot simply “turn off” the microphone above 4 kHz—audio sensors are analog continuous-time devices. You need an anti-aliasing FIR low-pass filter to remove high-frequency junk before downsampling and forwarding to the neural network.

Requirements: (1) Zero-phase response so speech timing is preserved (< 5 ms latency), (2) ≥ 60 dB attenuation above 8 kHz to suppress ultrasonic clicks, (3) no discontinuities in the passband (0–4 kHz) to avoid audible artifacts. Your Simulink model auto-generates optimized ARM Cortex-M4 code. Production devices ship to 10 million users annually.

1.2 Program 2: Cellular Modem Equalizers (Wireless Communications)

Your company supplies the signal processing firmware for cellular base stations and handsets. The critical challenge: wireless channels are not ideal low-pass filters. They introduce frequency-selective fading (some frequencies attenuate more than others) and multipath distortion (echoes from building reflections).

The engineering problem: A signal transmitted at 2.5 GHz arrives at the receiver distorted. The channel’s impulse response has multiple echoes. To recover the data, the receiver must apply an adaptive FIR equalizer—essentially the “inverse” of the channel—to flatten the frequency response and align multipath echoes.

Requirements: (1) Equalizer coefficients update in real-time (every 100 microseconds) to track channel changes, (2) 32-tap or 64-tap FIR filter running at 30.72 MHz sampling rate (LTE), (3) mean-squared-error between equalized output and reference falls below a strict threshold. Non-compliance breaks data throughput. Your Simulink-generated code runs on Qualcomm Snapdragon modems in 2 billion smartphones worldwide.

1.3 Program 3: Medical EEG Monitoring for Epilepsy Detection (Biomedical Devices)

Your company manufactures wearable EEG monitors for epilepsy patients. The device records brain electrical activity 24/7 and alerts patients to impending seizures.

The engineering problem: EEG signals are extraordinarily noisy. Muscle artifacts (EMG) from jaw clenching, eye blinks, and motion produce large-amplitude transients (1–100 Hz) that can mimic seizure-like activity. Electrical interference from power lines (60 Hz fundamental, harmonics at 120, 180, 240 Hz) also corrupts the recording. Meanwhile, true seizure activity is a high-amplitude, high-

frequency burst (typically 3–25 Hz for temporal lobe seizures).

Requirements: (1) Bandpass FIR filter passing 1–40 Hz (seizure band) while rejecting power-line interference and muscle artifacts, (2) Group delay < 100 ms (regulatory requirement for alert generation speed), (3) Clinical sensitivity $\geq 85\%$ and specificity $\geq 80\%$ (meaning the filter/detector pipeline must miss fewer than 15% of true seizures and falsely alarm on fewer than 20% of non-seizure events). Your Simulink model auto-generates code for an embedded ARM processor. Regulatory approval (FDA 510(k)) depends on hardware validation showing the filter meets these specs.

1.4 The Common Foundation: Model-Based Design with Simulink

All three programs use the same workflow:

1. **MATLAB design:** Hand-calculate or use `fir1()` to compute impulse response coefficients $h[n]$ for your specification (cutoff frequency, stopband attenuation, passband ripple).
2. **Simulink implementation:** Drag the Discrete FIR Filter block into your model, load coefficients from the MATLAB workspace, and simulate end-to-end performance.
3. **Code generation:** Press “Generate Code” and auto-generate optimized C/C++ ready for ARM, DSP, or FPGA hardware.
4. **Hardware validation:** Measure the empirical frequency response of the deployed code and compare against theory—this is how engineers certify designs for regulatory approval and high-volume manufacturing.

In this lab, you will design a prototype FIR filter, validate it with hardware measurements, and understand why the MATLAB/Simulink workflow is the industry standard for real-time DSP deployment.

2 Learning Objectives

By the end of this lab, you should be able to:

1. **Understand the Continuous-Time Fourier Transform (CTFT):** Recognize the sinc function as the impulse response of an ideal low-pass filter, and understand why real filters must approximate it.
2. **Design Finite Impulse Response (FIR) filters by hand:** Compute windowed-sinc coefficients for a specified cutoff frequency, then validate them against professional software.
3. **Use Model-Based Design in Simulink:** Build a real-time filter model graphically, insert the Discrete FIR Filter block, and load coefficients from the MATLAB workspace.
4. **Auto-generate embedded code:** Deploy a Simulink model to a microcontroller with a single click, replacing weeks of manual C++ coding.
5. **Measure frequency response empirically:** Sweep a sinusoidal input across the audio spectrum and measure input/output amplitude ratios to construct a Bode plot by hand.
6. **Validate theory with hardware:** Compare your measured frequency response to the theoretical curve computed by MATLAB's `freqz()` function and quantify accuracy.
7. **Understand real-world limitations:** Explain how ADC quantization, PWM timing, and component tolerances introduce deviations between ideal theory and practical hardware.

3 Core Concepts: FIR Filters and the Fourier Transform

3.1 The Ideal Low-Pass Filter and Its Impulse Response

An ideal low-pass filter with cutoff frequency ω_c (normalized to rad/sample) has a frequency response that is 1 (unity gain) below ω_c and 0 (total attenuation) above ω_c . The inverse Fourier transform of this ideal response yields the sinc function:

$$h_{\text{ideal}}[n] = \text{sinc}(n\omega_c/\pi) = \frac{\sin(n\omega_c)}{\pi n}, \quad (1)$$

with the convention that $h_{\text{ideal}}[0] = \omega_c/\pi$ (from L'Hôpital's rule).

The sinc function is non-causal (extends to $n = -\infty$) and infinite-length. Real filters approximate it by:

1. **Sampling:** Evaluate the sinc at discrete times $n = 0, \pm 1, \pm 2, \dots$
2. **Windowing:** Truncate to a finite length N (e.g., $N = 11$ coefficients).
3. **Centering:** Place the window symmetrically around $n = 0$ so the filter is nearly zero-phase.

3.2 Finite Impulse Response (FIR) Filters

An FIR filter is defined by the convolution equation:

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k] = h[n] * x[n], \quad (2)$$

where:

- $h[k]$ are the filter **coefficients** (or taps).
- N is the **filter order** (total number of taps).
- $x[n]$ is the input signal.
- $y[n]$ is the filtered output.
- The asterisk $*$ denotes convolution.

Key properties:

- **Linear:** Scaling the input scales the output proportionally.
- **Time-invariant:** Delaying the input delays the output by the same amount.
- **Causal:** The output at time n depends only on past and present inputs (no lookahead).
- **Stable:** If all coefficients are finite, the filter is always stable (bounded inputs produce bounded outputs).

3.3 Frequency Response of an FIR Filter

The frequency response is the Discrete-Time Fourier Transform (DTFT) of the impulse response:

$$H(e^{j\omega}) = \sum_{n=0}^{N-1} h[n]e^{-j\omega n}, \quad (3)$$

where ω is the normalized angular frequency (rad/sample), and $\omega \in [0, \pi]$ corresponds to the Nyquist band. The magnitude response $|H(e^{j\omega})|$ tells you how much each frequency is attenuated or amplified.

MATLAB's `freqz()` function computes this efficiently and plots the magnitude and phase responses.

3.4 Convolution in the Frequency Domain

The **convolution property** states that multiplication in the time domain is convolution in the frequency domain, and vice versa:

$$Y(e^{j\omega}) = H(e^{j\omega}) \cdot X(e^{j\omega}). \quad (4)$$

In plain English: the output spectrum is the product of the filter's frequency response and the input spectrum. High frequencies are attenuated if $|H(e^{j\omega})|$ is small at those frequencies.

[INSERT DIAGRAM: Ideal vs. Practical FIR Impulse Response (sinc function, windowing effect, side lobes)]

[INSERT DIAGRAM: Magnitude Response of Low-Pass FIR Filter (passband, transition zone, stopband, -3 dB cutoff)]

4 Required Equipment and Software

Table 1. Required Hardware and Software

Category	Required Items
Microcontroller	Arduino Uno (or compatible ARM Cortex microcontroller)
Audio Input	Electret microphone with preamplifier circuit, or function generator
ADC / Sampling	Arduino A0 pin (configured to 16 kHz sampling rate)
DAC / Output	Arduino Pin 9 (PWM output, 16 kHz carrier, RC low-pass filter)
Measurement	Oscilloscope (dual-channel, 20 MHz bandwidth typical)
Prototyping	Breadboard, jumper wires, resistor (10 k Ω), capacitor (100 nF)
Software	MATLAB R2021a or later with Signal Processing Toolbox
Software	Simulink with Embedded Coder, Arduino Support Package
Provided	<code>AudioFilter_Skeleton.slx</code> (pre-configured Simulink model)
Provided	Arduino boot-loader pre-installed on target microcontroller

Key Software Versions

- **MATLAB:** Must include Signal Processing Toolbox (for `fir1()` and `freqz()`).
- **Simulink:** Must have Embedded Coder and the Arduino Support Package installed (installed via `Support Package Installer`).
- **Arduino Board Support:** Simulink can target Arduino Uno, Due, or compatible boards running the Simulink Real-Time kernel.

5 Safety and Critical Notes

- **Audio Safety:** Keep microphone preamplifier output within the Arduino's ADC input range (0–5 V). Do not directly connect a line-level audio source (typically 1–2 V RMS or higher) without attenuation.
- **USB Power:** The Arduino is powered via USB. Ensure the USB cable is fully seated and the Arduino is recognized by the OS before opening Simulink.
- **Scope Probe Grounding:** Always connect oscilloscope probe ground to the breadboard ground rail to avoid ground loops and noise coupling.
- **Simulink Deployment:** Once the Simulink model is deployed, the microcontroller runs in real-time. Stop the model via the Simulink toolbar to halt execution and prevent continuous I/O.

6 Hardware Setup: The Audio Front-End

6.1 Signal Path Overview

The signal flows from a microphone (or function generator for testing) through a preamplifier, into the Arduino's ADC, through the digital FIR filter (running on the microcontroller), out through the Arduino's PWM pin, and finally to an oscilloscope for measurement:

Microphone / → Preamplifier → Arduino ADC (A0)
Function Gen (20 dB gain)

Arduino PWM (Pin 9) → RC Low-Pass Filter → Oscilloscope
(16 kHz carrier) (R=10k Ω , C=100nF) (smoothed output)

6.2 Microphone Preamplifier Circuit

A single electret microphone connected directly to the Arduino ADC would produce a very small signal (millivolts). The preamplifier stage provides approximately 20 dB of gain, amplifying the microphone signal to a level suitable for digitization.

**[INSERT MICROPHONE PREAMPLIFIER SCHEMATIC
(inverting amplifier stage, component values, pinout labels)]**

6.3 Arduino Connections

Table 2. Arduino Pin Assignments

Signal	Arduino Pin	Purpose
Microphone Input	A0 (ADC)	Reads amplified microphone signal at 16 kHz
Filtered Output	Pin 9 (PWM)	Outputs filtered signal at 16 kHz PWM frequency
Ground	GND	Common ground (breadboard and oscilloscope)

6.4 RC Low-Pass Filter on PWM Output

The Arduino's PWM output at Pin 9 is a square wave toggling at 16 kHz with a duty cycle controlled by the filter output value. To recover the analog waveform, a simple RC low-pass filter removes the high-frequency carrier:

Component values:

- Resistor: $R = 10\text{ k}\Omega$
- Capacitor: $C = 100\text{ nF}$
- Cutoff frequency: $f_{RC} = 1/(2\pi RC) \approx 159\text{ Hz}$

This is much lower than 16 kHz, so the PWM carrier is strongly attenuated and the filtered signal emerges as a smooth analog waveform.

**[INSERT RC FILTER AND OSCILLOSCOPE
CONNECTION SCHEMATIC (Arduino Pin 9 to R to Node
with C to GND, scope probes on both Ch1 and Ch2)]**

6.5 Oscilloscope Setup

- **Channel 1 (Yellow):** Probe connected after the RC filter (smoothed output).
- **Channel 2 (Blue):** Probe connected to the function generator or microphone preamplifier output (reference input).
- **Trigger:** Set to Channel 2, rising edge, auto mode.
- **Timebase:** $200\text{ }\mu\text{s}/\text{div}$ (for observing 100 Hz to 5 kHz signals).
- **Voltage Scale:** Ch1 and Ch2 both set to 1 V/div or 500 mV/div (adjust to fit the signal).

7 Pre-Lab Activity: Theoretical Filter Design

7.1 Assignment: Your Unique Cutoff Frequency

Your TA will assign each lab pair a **unique normalized cutoff frequency**. Record it here:

Student Notes: Your Assigned Cutoff Frequency:

$\omega_c =$ _____ rad/sample

Analog cutoff: $f_c =$ _____ Hz (where $f_s = 16$ kHz)

The relationship between normalized and analog cutoff is:

$$\omega_c \text{ (rad/sample)} = \frac{2\pi f_c}{f_s}, \quad (5)$$

where $f_s = 16000$ Hz is the sampling rate.

7.2 Task 1: Hand-Derive the Ideal Sinc Impulse Response

For your assigned ω_c , compute the ideal (infinite-length) impulse response at the specified sample indices. The sinc function is defined as:

$$\text{sinc}(u) = \begin{cases} 1 & u = 0 \\ \frac{\sin(\pi u)}{\pi u} & u \neq 0 \end{cases} \quad (6)$$

Your filter will have $N = 11$ coefficients centered at $n = 0$, so compute:

$$h[n] = \frac{\omega_c}{\pi} \text{sinc}\left(\frac{n\omega_c}{\pi}\right) \quad \text{for } n = -5, -4, \dots, 4, 5 \quad (7)$$

Pre-lab Deliverable (1/5): Show your hand calculations for at least three tap values (e.g., $h[0]$, $h[1]$, $h[2]$). Use a calculator if needed.

Student Notes: Hand-Calculated Sinc Coefficients:

7.3 Task 2: List Your Windowed FIR Coefficients

For a rectangular window of length $N = 11$, your hand-derived sinc values *are* your final windowed coefficients. Record them as a vector:

Pre-lab Deliverable (2/5): List all 11 coefficients (from $h[-5]$ to $h[5]$).

Student Notes: Windowed FIR Coefficients (N=11):

7.4 Task 3: Compare with MATLAB's `fir1()`

In MATLAB, run the following code to generate the professional filter coefficients:

```
% Pre-Lab Filter Design
Nfir = 11; % Number of taps
omega_c_rad_sample = 0.4 * pi; % YOUR ASSIGNED VALUE (in rad/sample)
normalized_freq = omega_c_rad_sample / pi; % Normalize to [0, 1]

% Generate FIR filter using MATLAB's fir1()
h_filter = fir1(Nfir - 1, normalized_freq);
```

```
fprintf('MATLAB□fir1()□coefficients:\n');
disp(h_filter);

% Compute and plot frequency response
[H, w] = freqz(h_filter, 1, 512);
f = w / (2*pi) * 16000; % Convert to Hz
H_mag = abs(H);

figure;
semilogy(f, H_mag);
xlabel('Frequency□(Hz)');
ylabel('Magnitude□|H(e^{j\omega})|');
title('FIR□Filter□Frequency□Response□(MATLAB□fir1)');
grid on;
```

Pre-lab Deliverable (3/5): Record the MATLAB-generated coefficients and create the frequency response plot.

Student Notes: MATLAB fir1() Output:

7.5 Task 4: Compare Hand vs. MATLAB

Compute the absolute difference between your hand-derived coefficients and MATLAB's output. Are they close?

Pre-lab Deliverable (4/5): Show the coefficient comparison table (hand value, MATLAB value, absolute difference). Typical discrepancy should be < 0.01 .

Student Notes: Coefficient Comparison (Hand vs. MATLAB):

8 Software Implementation: Simulink Model-Based Design

8.1 Overview of the Skeleton Model

A file named `AudioFilter_Skeleton.slx` has been pre-configured with:

- **ADC Input Block:** Reads from Arduino A0 at 16 kHz sampling rate.
- **Discrete FIR Filter Block:** (Currently empty; you will add coefficients.)
- **DAC Output Block:** Writes to Arduino Pin 9 at 16 kHz PWM frequency.
- **Model Settings:** Target microcontroller, sample time, and hardware configuration are pre-set.

Your job: Insert your FIR coefficients into the Discrete FIR Filter block and deploy.

8.2 Step 1: Prepare Your MATLAB Workspace

Before opening Simulink, ensure your FIR coefficients are in the MATLAB base workspace:

```
% Prepare FIR coefficients for Simulink
Nfir = 11;
omega_c_rad_sample = 0.4 * pi; % YOUR ASSIGNED VALUE
normalized_freq = omega_c_rad_sample / pi;
h_filter = fir1(Nfir - 1, normalized_freq);

% Make sure this variable is in the base workspace
assignin('base', 'h_filter', h_filter);
disp('h_filter saved to workspace.');
```

8.3 Step 2: Open the Skeleton Simulink Model

```
open('AudioFilter_Skeleton.slx');
```

Simulink will open a new window showing the model.

[INSERT SIMULINK MODEL SCREENSHOT (showing ADC, empty FIR block, DAC blocks before coefficient insertion)]

8.4 Step 3: Insert the Discrete FIR Filter Block

1. In the Simulink model, locate the **Discrete FIR Filter** block (should be pre-placed as a placeholder).
2. Double-click the block to open its parameters dialog.

**[INSERT SCREENSHOT OF DISCRETE FIR FILTER
BLOCK PARAMETER DIALOG]**

8.5 Step 4: Load Coefficients from Workspace

In the Discrete FIR Filter block parameters:

- **Coefficient source:** Select “Specify via dialog”.
- **Coefficients:** In the text field, enter `h_filter` (the variable name from your MATLAB workspace).
- Click **OK** to apply.

Simulink will read the variable from the base workspace and use those 11 coefficients.

8.6 Step 5: Build, Deploy & Start

1. In the Simulink toolbar, click “**Monitor & Tune**” (or select **Simulation** → **Deploy to Hardware**).
2. Simulink will automatically:
 - (a) Compile your model to optimized C++ code.
 - (b) Build the executable for the Arduino.
 - (c) Flash the binary onto the microcontroller via USB.
 - (d) Start real-time execution.
3. Watch the MATLAB Command Window for build messages. You should see “Build Successful” and Arduino LEDs will blink during flashing.
4. The model enters real-time mode; your FIR filter is now running on the hardware.

CRITICAL: Do not modify the ADC/DAC blocks or sample rate. These are pre-configured and locked. Only change the FIR Filter coefficients.

**[INSERT SCREENSHOT OF COMPLETED SIMULINK
MODEL (showing all three blocks connected, FIR block
labeled with coefficients)]**

9 Measurement & Verification: Empirical Frequency Response

9.1 Measurement Strategy

You will measure your filter's frequency response by sweeping a sinusoidal input across the audio spectrum and recording input/output amplitude ratios. This is called a **Bode plot by hand**.

9.2 Equipment Configuration

- **Function Generator:** Set to sinusoidal output, approximately 1 V peak amplitude.
- **Oscilloscope Channel 2:** Probe the function generator (reference signal).
- **Oscilloscope Channel 1:** Probe the Arduino's RC-filtered output (filtered signal).
- Both channels set to 1 V/div or 500 mV/div for clear visibility.

9.3 Task 1: Measure Gain at Key Test Frequencies

Sweep the function generator through the following frequencies and record input/output peak-to-peak voltages:

Student Notes: Frequency Response Measurements:

Freq (Hz)	$V_{in,pp}$ (V)	$V_{out,pp}$ (V)	Gain	Gain (dB)
100				
500				
1000				
f_c (your cutoff)				
2000				
4000				
6000				
8000				

At each frequency:

1. Set function generator to the desired frequency.
2. Let oscilloscope display stabilize (10–20 cycles).
3. Measure peak-to-peak voltage of Ch2 (input reference).
4. Measure peak-to-peak voltage of Ch1 (filtered output).
5. Compute gain: $\text{Gain} = V_{out,pp}/V_{in,pp}$.
6. Compute gain in dB: $\text{Gain}_{dB} = 20 \log_{10}(\text{Gain})$.

9.4 Task 2: Capture Scope Waveforms at Key Frequencies

Checkpoint (2/6): Photograph or screenshot the oscilloscope display at three key frequencies:

1. **Passband (100 Hz):** Input and output should be nearly identical in amplitude.

[INSERT SCOPE CAPTURE AT 100 HZ (passband,
minimal attenuation)]

2. **Cutoff Frequency (f_c):** Output should be attenuated to approximately 70.7% of input amplitude (-3 dB point).

[INSERT SCOPE CAPTURE AT CUTOFF FREQUENCY
(output $\approx 0.707 \times$ input)]

3. **Stopband (6000–8000 Hz):** Output should be significantly attenuated compared to input.

[INSERT SCOPE CAPTURE AT HIGH FREQUENCY
(strong attenuation in stopband)]

9.5 Task 3: Create the Empirical Bode Plot in MATLAB

Plot your measured frequency response data alongside the theoretical curve. Use MATLAB's `freqz()` to compute the theory:

```
% Empirical Frequency Response Plot  
% Your measured data
```

```
f_measured = [100, 500, 1000, f_c, 2000, 4000, 6000, 8000];
gain_measured = [...]; % Fill in your measured gain values (linear, not dB)

% Theoretical response from MATLAB
[H_theory, w] = freqz(h_filter, 1, 1024);
f_theory = w / (2*pi) * 16000; % Convert to Hz
H_mag_theory = abs(H_theory);

%% Plot magnitude response
figure('Position', [100, 100, 1000, 700]);
subplot(2, 1, 1);
semilogy(f_theory, H_mag_theory, 'b-', 'LineWidth', 2);
hold on;
semilogy(f_measured, gain_measured, 'ro', 'MarkerSize', 10, ...
         'LineWidth', 2);
xlabel('Frequency (Hz)');
ylabel('Magnitude |H(e^{j\omega})|');
legend('Theory (freqz)', 'Hardware Measurement', 'Location', 'best');
title(['FIR Filter Frequency Response: f_c = ' num2str(f_c) ' Hz']);
grid on;
xlim([0, 10000]);

%% Plot phase response
subplot(2, 1, 2);
phase_theory = angle(H_theory) * 180 / pi;
plot(f_theory, phase_theory, 'b-', 'LineWidth', 2);
xlabel('Frequency (Hz)');
ylabel('Phase (degrees)');
grid on;
xlim([0, 10000]);

%% Add name and student ID
sgtitle(['Lab 09 Frequency Response | Your Name | Student ID']);
```

Pre-lab Deliverable (5/5): Generate the empirical frequency response plot (magnitude and phase) and include your name/student ID in the title.

9.6 Task 4: Analyze Agreement Between Theory and Hardware

Discussion (1/1): Answer the following questions:

Student Notes: Analysis Questions:

1. How close are your measured points to the theoretical curve? Estimate maximum error in dB.
2. Does your filter achieve the expected -3 dB attenuation at f_c ?
3. What is the observed attenuation (in dB) in the stopband (6–8 kHz)?
4. List potential sources of error (ADC quantization, PWM jitter, component tolerances, etc.).

10 Deliverables & TA Sign-Off

Four Checkpoints for Progressive Assessment

Your lab is graded at four checkpoints. Your TA will initial each one and record the time. Work efficiently to finish within the 2-hour window.

Checkpoint 1: Filter Design Validation (Target: 20 min)

Checkpoint (3/6): Before construction, show the TA:

- ☐ Pre-lab hand-derived windowed-sinc coefficients (from Pre-Lab Tasks 1 and 2).
- ☐ MATLAB `fir1()` output for the same filter order (from Pre-Lab Task 3).
- ☐ Comparison table showing absolute differences between hand and MATLAB values (from Pre-Lab Task 4).
- ☐ TA verifies: coefficients are mathematically correct and differences are within tolerance (< 0.01 per tap).

TA Signature: _____ **Time:** _____

Checkpoint 2: Simulink Model and Deployment (Target: 30 min)

Checkpoint (4/6): Before measurement, show the TA:

- ☐ Screenshot of the completed Simulink model (showing ADC $\rightarrow$ FIR Filter $\rightarrow$ DAC blocks properly connected).
- ☐ Discrete FIR Filter block parameters showing `h_filter` variable loaded.
- ☐ Evidence of successful build and deployment: MATLAB Command Window showing “Build Successful” and/or Arduino LED activity.
- ☐ TA verifies model is running in real-time on hardware and accepts function generator input.

TA Signature: _____ **Time:** _____

Checkpoint 3: Scope Captures and Measurement (Target: 35 min)

Checkpoint (5/6): Show the TA:

- ☐ Three scope captures (photographs or USB exports) at 100 Hz, f_c , and 6–8 kHz showing input and output signals.
- ☐ A completed measurement table with at least 8 frequency points, peak-to-peak voltages, and computed gains.
- ☐ Observed behavior matches expectations: minimal attenuation in passband (≥ 1 dB), ≈ -3 dB at f_c , strong attenuation (≥ 10 dB) in stopband.
- ☐ TA verifies hardware is functioning correctly and measurements are physically plausible.

TA Signature: _____ **Time:** _____

Checkpoint 4: Empirical Frequency Response Plot (Target: 25 min)

Checkpoint (6/6): Show the TA:

- ☐ MATLAB plot showing both magnitude and phase responses.
- ☐ Theoretical curve (blue line) computed with `freqz()` over the range 0–10 kHz.
- ☐ Measured hardware data points (red markers) overlaid on the theoretical curve.
- ☐ Proper axis labels: Frequency (Hz), Magnitude (linear or dB), Phase (degrees).
- ☐ Your name and student ID in the figure title (`sgtitle`).
- ☐ TA verifies: agreement within engineering tolerance (± 3 dB in passband, ± 5 dB in stopband).

TA Signature: _____ **Time:** _____

Final Grade Summary

Checkpoint	Points	Signed Off
1. Filter Design Validation	25	☐
2. Simulink Model & Deployment	25	☐
3. Scope Captures & Measurements	25	☐
4. Frequency Response Plot	25	☐
TOTAL	100	

Final TA Signature: _____ **Final Grade:** _____

A Fourier Transform Fundamentals

A.1 Continuous-Time Fourier Transform (CTFT)

The CTFT of a continuous-time signal $x(t)$ is defined as:

$$X(j\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt, \quad (8)$$

and the inverse CTFT is:

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(j\omega)e^{j\omega t} d\omega. \quad (9)$$

The CTFT maps time-domain signals to frequency-domain representations, revealing the spectral content (which frequencies are present and at what amplitudes).

A.2 The Ideal Low-Pass Filter

An ideal low-pass filter has a rectangular frequency response:

$$H_{\text{ideal}}(j\omega) = \begin{cases} 1 & |\omega| < \omega_c \\ 0 & |\omega| \geq \omega_c \end{cases} \quad (10)$$

Applying the inverse CTFT yields the sinc impulse response:

$$h_{\text{ideal}}(t) = \frac{\omega_c}{\pi} \text{sinc}\left(\frac{\omega_c t}{\pi}\right). \quad (11)$$

This ideal response extends from $t = -\infty$ to $t = \infty$ and is thus non-realizable in practice. Real filters approximate this ideal by windowing and truncating the sinc function.

B Discrete-Time Fourier Transform and FIR Filters

B.1 Discrete-Time Fourier Transform (DTFT)

For a discrete-time signal $x[n]$, the DTFT is:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n}, \quad (12)$$

where $\omega \in [0, 2\pi)$ is the normalized angular frequency. The DTFT is periodic with period 2π .

B.2 FIR Filter Design: Window Method

The most straightforward design method:

1. Start with the ideal sinc impulse response.
2. Truncate to N coefficients centered at $n = 0$.

3. Optionally apply a window function (rectangular, Hamming, Hann) to reduce side lobe artifacts.

MATLAB's `fir1()` implements this using a Hamming window by default, which provides good frequency selectivity and low ripple in both passband and stopband.

C MATLAB Code Templates and Examples

C.1 Complete Pre-Lab Workflow

```
%% Lab 09 Pre-Lab: FIR Filter Design
clear; clc;

%% Assigned parameters
omega_c_rad_sample = 0.4 * pi; % CHANGE TO YOUR ASSIGNED VALUE
fs = 16000; % Sampling rate in Hz

%% Compute analog cutoff frequency
f_c = (omega_c_rad_sample / (2*pi)) * fs;
fprintf('Lab 09 Pre-Lab\n');
fprintf('Cutoff (rad/sample): %.4f\n', omega_c_rad_sample);
fprintf('Cutoff (Hz): %.1f\n\n', f_c);

%% Task 1: Hand-derive sinc coefficients
n = -5:5;
h_hand = sinc(n .* omega_c_rad_sample / pi);
fprintf('Hand-derived sinc coefficients:\n');
fprintf('n=%d: h[n]=%.6f\n', [n; h_hand]);

%% Task 3: MATLAB fir1()
Nfir = 11;
normalized_freq = omega_c_rad_sample / pi;
h_filter = fir1(Nfir - 1, normalized_freq);
fprintf('\nMATLAB fir1() coefficients:\n');
fprintf('n=%d: h[n]=%.6f\n', [n; h_filter]);

%% Frequency response
[H, w] = freqz(h_filter, 1, 1024);
f = w / (2*pi) * fs;
H_mag = abs(H);
H_phase = angle(H) * 180 / pi;

figure('Position', [100, 100, 1000, 600]);
subplot(2, 1, 1);
semilogy(f, H_mag, 'b-', 'LineWidth', 2);
xlabel('Frequency (Hz)');
ylabel('Magnitude |H(e^{j\omega})|');
title(sprintf('FIR Filter: f_c = %.1f Hz', f_c));
grid on;
xlim([0, 8000]);

subplot(2, 1, 2);
plot(f, H_phase, 'b-', 'LineWidth', 2);
xlabel('Frequency (Hz)');
```

```
ylabel('Phase (degrees)');  
grid on;  
xlim([0, 8000]);
```

D Simulink and Model-Based Design Guide

D.1 Setting Up the Arduino Target

- Open Simulink and select **Simulation** → **Model Configuration Parameters**.
- Navigate to **Hardware Implementation**.
- Set **Hardware Board** to “Arduino Uno”.
- Verify that the **Solver** is set to “Fixed-step”, “Discrete (no continuous states)”.

D.2 The Discrete FIR Filter Block

Location in Simulink Library Browser: **Simulink** → **Filtering** → **Discrete Filters**.

Key Parameters:

- **Coefficients:** Reference a MATLAB workspace variable (e.g., `h_filter`).
- **Sample Time:** Set to 1/16000 s (matching the ADC sample rate).
- **Initial Conditions:** Leave as default (zeros).

E Troubleshooting Guide

E.1 Pre-Lab Issues

- **fir1() not available:** Verify Signal Processing Toolbox is installed. Run `ver` in MATLAB to check.
- **Sinc values disagree with MATLAB:** Double-check the sinc definition. MATLAB uses $\text{sinc}(x) = \sin(\pi x)/(\pi x)$ with $\text{sinc}(0) = 1$.
- **Coefficients very different from MATLAB output:** Verify your normalized frequency is in $[0, 1]$. Compare with `fir1(10, omega_c/pi)` in MATLAB.

E.2 Simulink Deployment Issues

- **Arduino not recognized:** Check USB cable and ensure Arduino board shows in Device Manager (Windows) or `/dev/` (Mac/Linux). Try “Build Hardware” without deployment first.
- **Build fails with unknown symbol:** Ensure `h_filter` variable is in the MATLAB base workspace. Run `assignin('base', 'h_filter', h_filter);` before deployment.
- **ADC reads all zeros:** Check that A0 is connected to the preamplifier output and powered. Verify Arduino can read analog voltages independently (e.g., with a simple sketch).
- **No output on Pin 9:** Verify the DAC output block is properly configured. Check Pin 9 with a multimeter (should see toggling PWM).

E.3 Measurement Issues

- **Scope shows very noisy signal:** Verify oscilloscope probes are properly grounded. Use short, twisted probe leads. Increase averaging if available.
- **Measured attenuation is much less than expected:** Check that the correct coefficients were loaded into the FIR block. Re-deploy the model if coefficients changed.
- **Frequency response doesn't match theory:** ADC/DAC quantization and PWM timing jitter cause deviations. These are normal; document and explain in your analysis.

References and Inspiration

University Course Inspirations

This lab represents a convergence of signal theory, practical DSP implementation, and professional software engineering:

- **MIT 6.003 (Signals, Systems, and Inference):** The derivation of the sinc function as the ideal low-pass impulse response, the Discrete-Time Fourier Transform (DTFT), and the frequency response of FIR filters are foundational to this lab. MIT's rigorous mathematical treatment of signal analysis informs the windowed-sinc design methodology.
- **MIT 6.007 (Applied Electromagnetics):** While this lab focuses on digital filters, the underlying circuit concepts (passband ripple, stopband attenuation, phase response) originate in analog filter theory, which MIT's circuits curriculum emphasizes.
- **Rice University ELEC 241 (Real-Time DSP):** Rice's foundational expertise in embedded, real-time signal processing is the heartbeat of this lab. ELEC 241 emphasizes **direct implementation**: code that actually runs on microcontrollers, not just simulation. Rice's philosophy is reflected in the Simulink-to-embedded-code workflow—the method by which professional DSP engineers deploy filters to billions of devices. The lab's focus on finite arithmetic, group delay, and trade-offs between filter order and computational cost directly mirrors ELEC 241's pedagogical priorities.
- **Georgia Tech ECE 2026 (Real-Time DSP):** Georgia Tech's strength in audio processing and real-time system validation informs the lab's focus on empirical frequency response measurement and hardware validation. The three application contexts (hearing aids, cellular modems, EEG monitoring) reflect Georgia Tech's breadth of DSP applications.
- **Duke University ECE 280:** This lab bridges the continuous-time Fourier analysis of Labs 1–7 with discrete-time signal processing and embedded implementation. Students progress from analog prototype (Lab 08) to digital realization (Lab 09)—a fundamental engineering workflow.

Core References

- Oppenheim, A. V., Schafer, R. W., & Buck, J. R. (2010). *Discrete-Time Signal Processing* (3rd ed.). Prentice Hall.
 - **Chapters 5–7:** Discrete-Time Fourier Transform, FIR filter design, windowing, and the frequency response of digital filters.
 - **Standard reference:** Textbook used in undergraduate DSP courses worldwide.
- Harris, F. J. (1978). "On the use of windows for harmonic analysis with the discrete Fourier transform." *Proceedings of the IEEE*, 66(1), 51–83.
 - **Foundational paper:** Explains why windows are necessary for FIR filter design and catalogs trade-offs between window families (rectangular, Hamming, Hann, Blackman).
 - **Practical focus:** The Hamming window you use in this lab is justified by this seminal work.
- Orfanidis, S. J. (2009). *Introduction to Signal Processing*. Prentice Hall (available online at www.ece.rutgers.edu/~orfanidi/intro2sp/).

- **Chapters 10–11:** FIR and IIR filter design with detailed windowed-sinc methodology.
- **Clear derivations:** Explains the sinc function and windowing with pedagogical clarity suitable for undergraduate students.
- The MathWorks. *Simulink User Guide and Code Generation Documentation*.
 - **Professional toolchain:** Simulink is the industrial standard for Model-Based Design in automotive, aerospace, and consumer electronics.
 - **Code generation:** The Embedded Coder and Arduino Support Package transform your graphical model into optimized C code deployed to embedded processors.
- Zölzer, U. (2011). *DAFX: Digital Audio Effects* (2nd ed.). Wiley.
 - **Audio-specific:** Real-world audio filter design, including hearing aids, equalizers, and noise reduction—three of the application contexts in this lab.
 - **Hardware constraints:** Addresses fixed-point arithmetic, latency, and real-time CPU budgets.
- Steiglitz, K. (1996). *A Digital Signal Processing Primer: With Applications to Digital Audio and Computer Music*. Addison-Wesley.
 - **Intuitive approach:** Connects digital filter theory to audible sound and musical instruments, making the abstract concrete.
- IEEE 754-2019: *IEEE Standard for Floating-Point Arithmetic*.
 - **Hardware constraint:** Defines how real numbers are represented in embedded processors and DSP cores. Finite precision impacts filter stability and coefficient quantization.

Industry Standards and Validation

- IEEE 1057: *Standard for Digitizing Waveform Recorders*.
 - **Measurement standard:** Defines best practices for empirically measuring frequency response of digital systems in hardware.
 - **Hardware validation:** This lab’s methodology of sweeping sinusoids and measuring amplitude ratios follows IEEE 1057 principles.
- FDA 510(k) Predicate Devices (biomedical context):
 - **Regulatory requirement:** For the EEG monitoring application (Program 3), the filter must be validated both in simulation and hardware. Clinical sensitivity and specificity benchmarks are FDA-mandated.
- 3GPP TS 36.104 (LTE Specification):
 - **Wireless communications context:** For the cellular modem equalizer application (Program 2), channel equalization performance is governed by 3GPP standards. Your FIR filter’s coefficients must adapt to maintain signal-to-interference-plus-noise ratio (SINR) above a threshold.

Software Tools and Deployment

- MATLAB & Simulink (MathWorks).
 - `fir1()`: Compute windowed-sinc coefficients (Hamming, Hann, Blackman, Kaiser windows).
 - `freqz()`: Compute and plot the frequency response of an FIR filter.
 - Simulink Discrete FIR Filter block: Real-time filter implementation.
 - Embedded Coder: Auto-generate optimized C/C++ code for ARM Cortex-M, DSP, and FPGA targets.
- Arduino IDE with Simulink Support Package.
 - **Microcontroller target:** Deploy Simulink models to Arduino Uno, Mega, and MKR boards.
 - **Real-time sampling:** Achievable ADC rates up to 16 kHz (sufficient for audio and biomedical applications).
- GNU Octave (free alternative to MATLAB).
 - **Open-source:** Equivalent `fir1()` and `freqz()` functions available in Octave's Signal Package.

Why This Lab Bridges Theory and Practice

This lab teaches the **professional DSP workflow**: from mathematical theory (sinc function, Fourier transforms) to graphical design (Simulink blocks) to auto-generated embedded code. This is how every smartphone, hearing aid, and base station filter is designed and validated. By implementing this workflow yourself—deriving coefficients by hand, building a Simulink model, and measuring hardware performance—you gain the intuition and skills that make you a productive DSP engineer in industry.